

A Story of a Painful Migration: Modernizing a Java Monolith with Agentic AI

Marek Dominiak



Marek Dominiak

I run training in EventStorming, Architecture, DDD, and TDD

19 years of professional experience

Co-owner and instructor at 

Hands-on Architect
and Team Lead at



<https://www.linkedin.com/in/marekdominiak>

Agenda

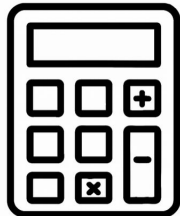
- Context of the migration
- Plan for migration (Theory and Practice)
- Easy and Hard Migrations
- Benefits
- Agentic AI
- Lessons learned
- Q/A



Context of the migration

Numbers

- The system developed over the last 16 years
- Team size ~25-30 people during this time
- ~13 different services
- Number of files: ~64K
- Lines of code: > 2M
- Number of user accounts: ~ 3M



Why did we need to upgrade?

- Bad Developer Experience
- Increased cost of keeping things secure
- Benefits from JVM upgrade
- Costs related to running outdated versions on Cloud (e.g. Mysql on AWS)



Max CVSS	9.8
EPSS Score	49.28%
Published	2018-01-04
Updated	2022-04-07

Max CVSS	9.8
EPSS Score	6.74%
Published	2023-04-20
Updated	2023-08-28



What did hold us back?

The system is
wide and deep

New projects

New integrations

Fear of Big Bang
Release

Slow adoption of
SPA admin
panel

Keeping system
uptime ~100%

Upgrade: In Theory

In Theory



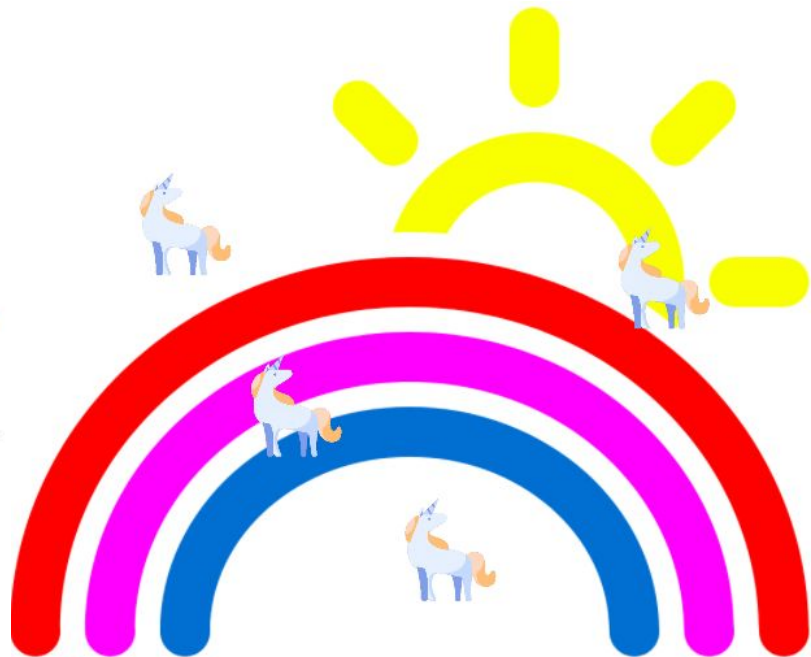
Vlad Larichev • 2nd

Scaling Industrial AI with Impact | Engineer & Software Developer ...

9mo • 🌐

+ Follow

Impressive: Amazon just upgraded over 50% of their Legacy Java Code with their **#GenAI** tool Amazon Q, saving **4,500 Developer-Years** (!) and \$260M annually.



What's the difference between Theory and Practice?

In theory, there is no difference -
but in practice, there is.

Theory vs Practice

- In just a few minutes/hours we should be done.

- Too good to be true?

- Yes, it is :)



vas

@vasumanmoza



Claude 4 just refactored my entire codebase in one call.

25 tool invocations. 3,000+ new lines. 12 brand new files.

It modularized everything. Broke up monoliths. Cleaned up spaghetti.

None of it worked.

But boy was it beautiful.

- Using AI for our upgrade would lead to more time spent than doing it by ourselves.

Ref: <https://x.com/vasumanmoza/status/1926487201463832863>

Ref 2: https://www.reddit.com/r/aws/comments/1jhij9/aws_q_was_great_until_it_started_lying/

Ref 3: https://www.perplexity.ai/page/replit-ai-agent-deletes-user-s-1w_FZlpCQDiCop8A6V_mtg

Ref 4: <https://medium.com/@sobyx/the-ais-existential-crisis-an-unexpected-journey-with-cursor-and-gemini-2-5-pro-7dd811ba7e5e>

Upgrade: In Practice

Parts where migration was easy

“Easy” upgrades

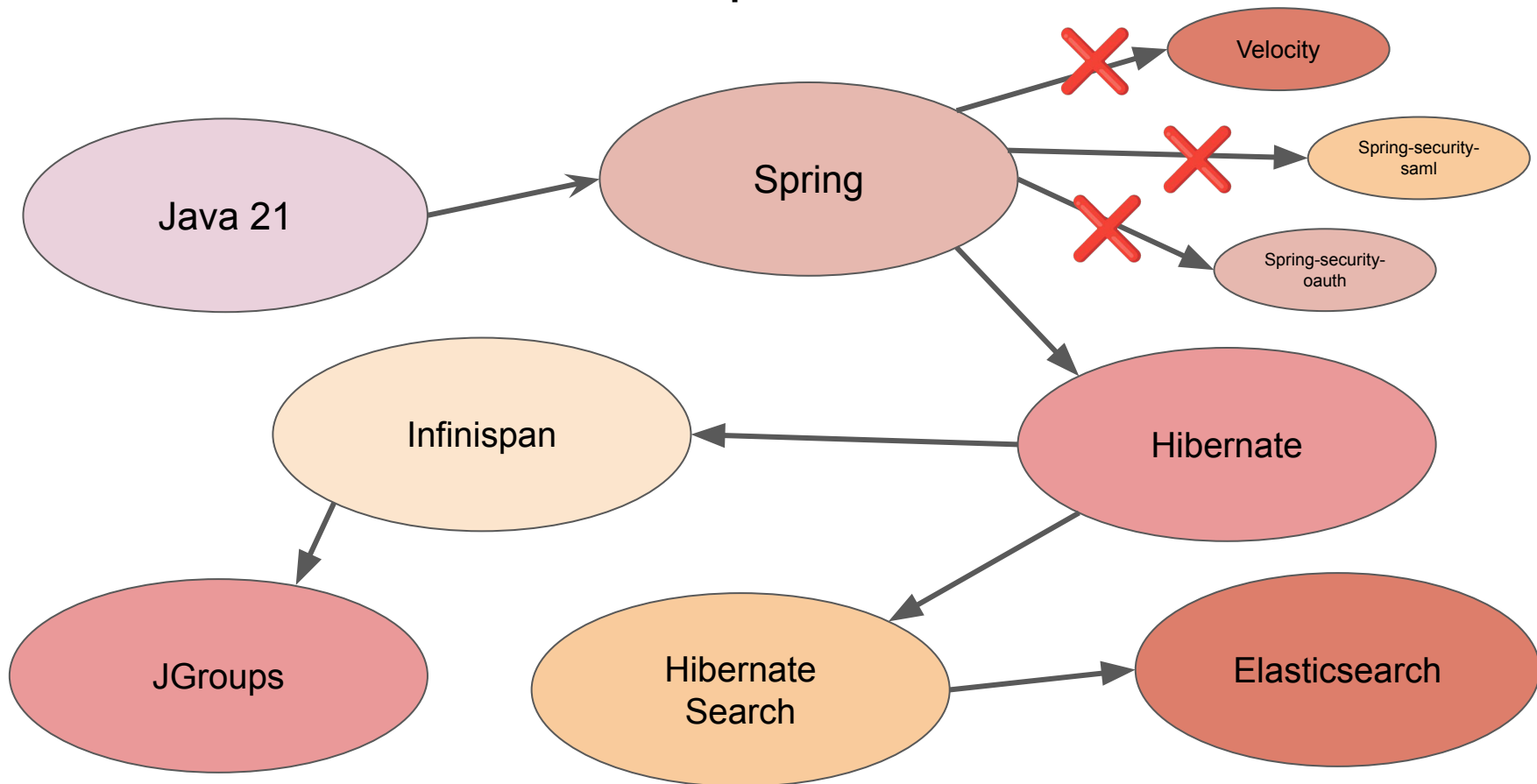
- Java 11 to Java 21
- Tomcat 9 to Tomcat 11
- Spring Framework (Core, MVC) 4.3.x to 6.x



Parts where migration was NOT easy

Challenge 1 - Chained Dependencies

Chained Dependencies

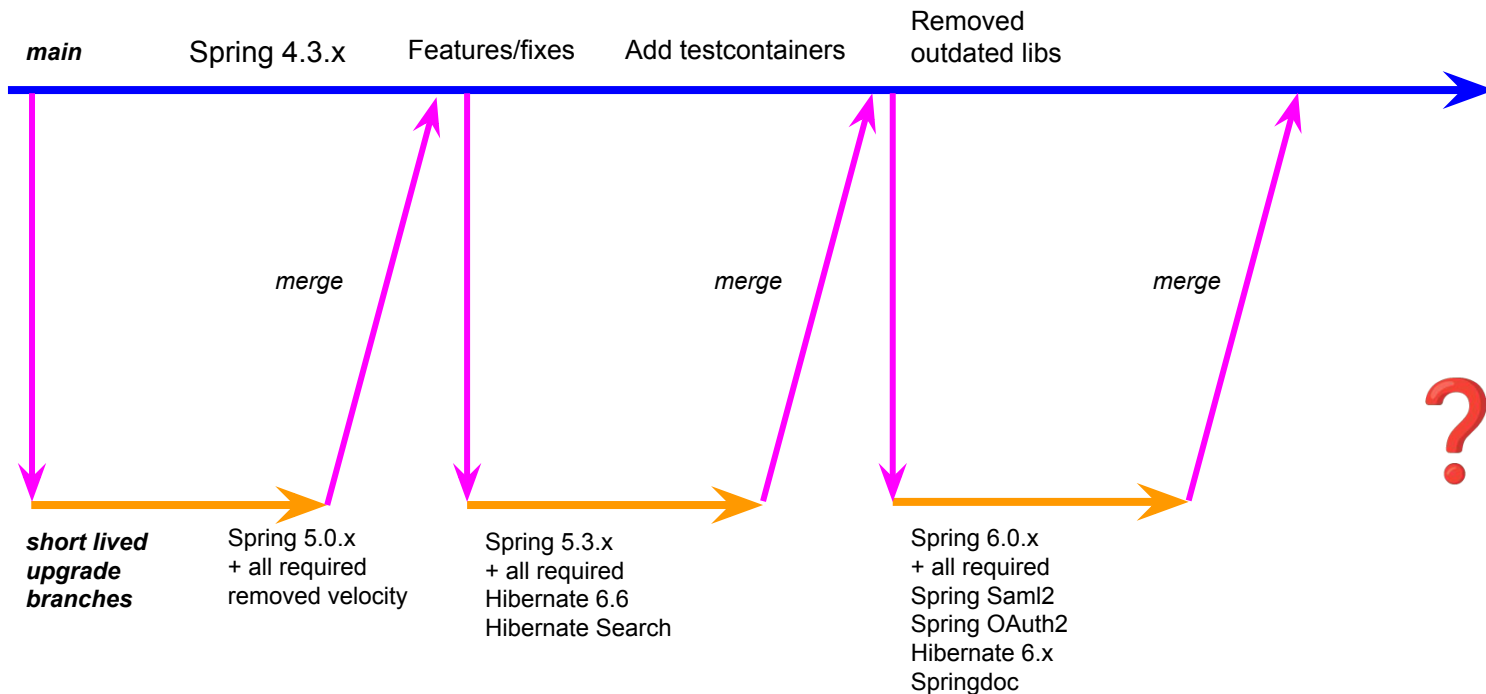


What were our options for upgrade?

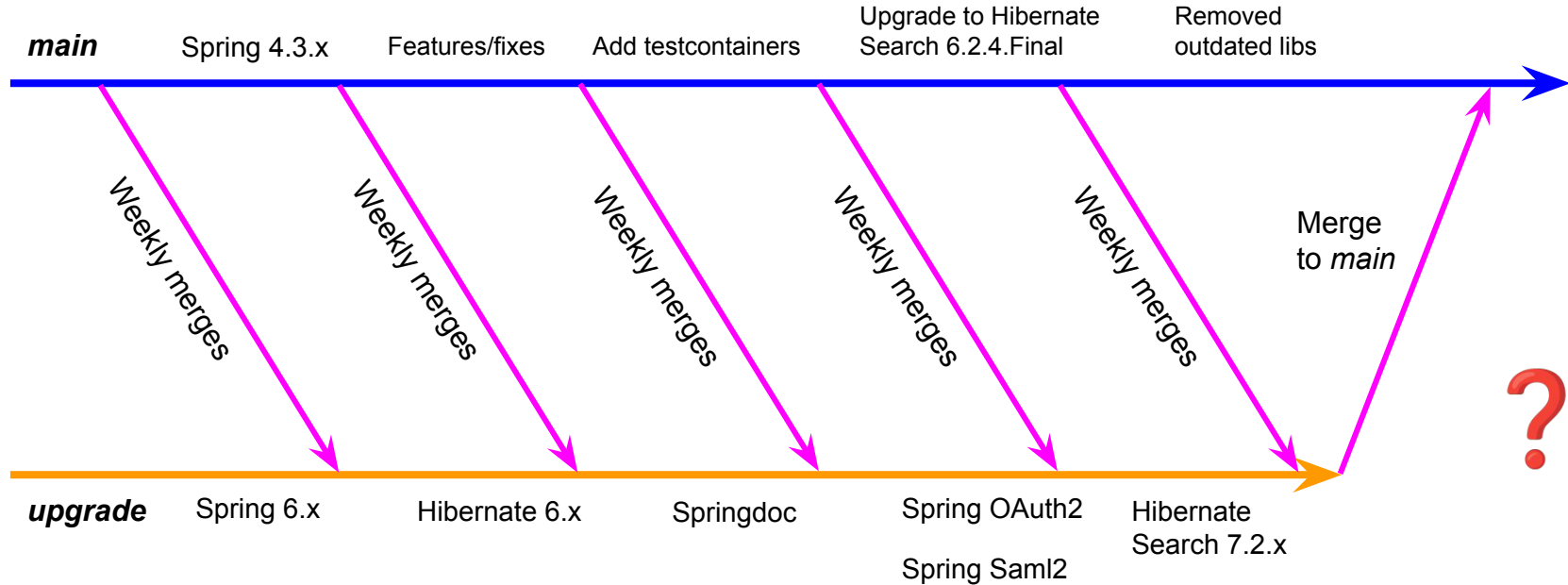
1. Gradual upgrade from:
 - a. Spring 4.3.x to Spring 5.0.x
 - b. Spring 5.0.x to Spring 5.3.x
 - c. Spring 5.3.x to Spring 6.0.x.
 - d. **Short-lived branches**, same with other frameworks.

2. Long lived branch for upgrade from **Spring 4.3.x** to **Spring 6.x**, all libs and frameworks in one sweep.

Options for upgrade 1/2 (main + “short” lived branches)



Options for upgrade 2/2 (main + long lived remote branch)

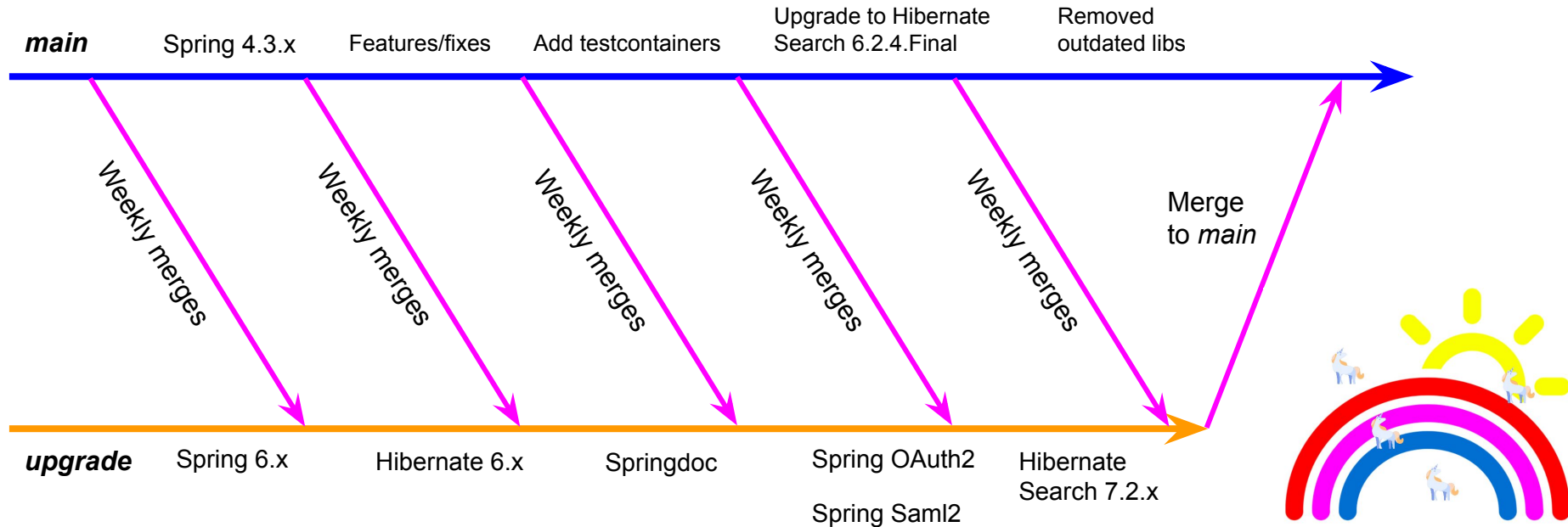


What did we do?

- Because:
 - There were **6 major/minor** releases between 4.3.x and 6.0.x, and **hundreds** of fixed issues.
 - We would have been affected by those issues during the time between 4.3.x and 6.0.x.
 - It would take more time and might not even be possible.
 - So choosing **Option 1 (short lived branches)** was **NO - GO.** ❌



We chose option 2

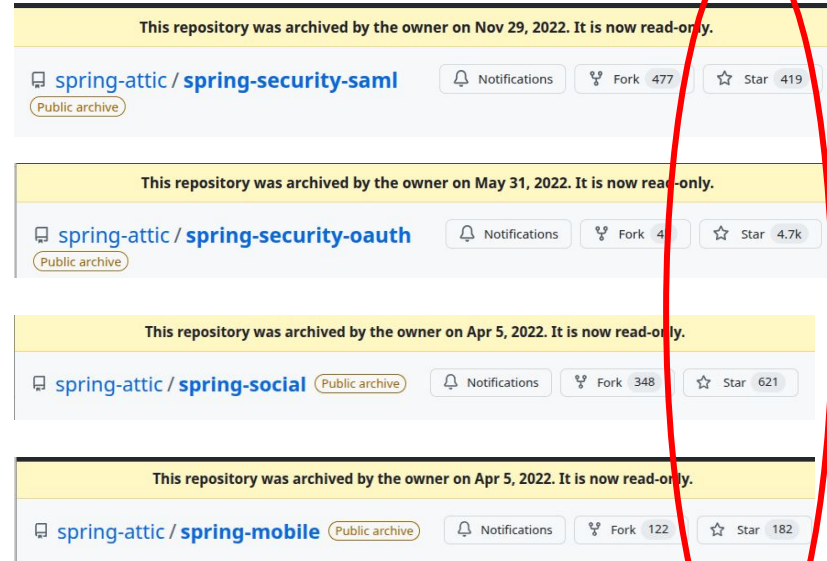


Challenge 2 - Libraries that disappeared



Libraries that disappeared

- Spring-security-saml
- Spring-security-oauth
- Spring-social
- Spring-mobile
- Springockito
- Hystrix
- Fongo
- Spring-Velocity



Hystrix Status

Hystrix is no longer in active development, and is currently in maintenance mode.

Libraries that disappeared - replacements

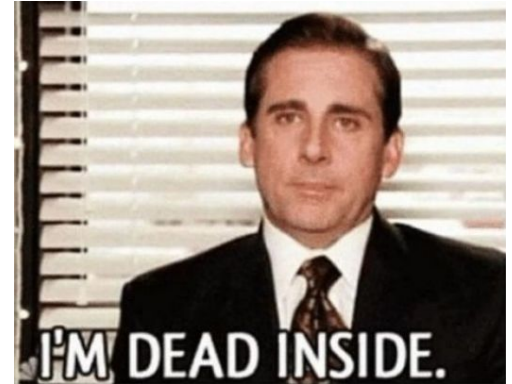
- `Spring-security-saml` → `spring-security-saml2-service-provider`
- `Spring-security-oauth` → `spring-security-oauth2-client`
- `Spring-social` → removed in the main branch
- `Spring-mobile` → removed in the main branch
- `Springockito` → we removed it and replaced with custom implementation of a `TestExecutionListener`, as `@MockBean` isn't available in the base Spring
- `Hystrix` → `resilience4j`
- `Fongo` → `testcontainers`
- `Spring-velocity` → removed, replaced with `SPA`, and `freemaker`



Challenge 3 - Changed APIs

Changed APIs

- *javax* → *jakarta*
- Hibernate / Hibernate Search
- Spring Security Core
- Infinispan
- Mockito / PowerMock removal
- Apache Tiles / Spring WebFlow
- Apache HttpClient / HttpCore
- Many, many more ...



Spring - summary

- Spring 4 → 6: not one upgrade, but many small traps
- javax → jakarta was easy only on surface
- Spring Security changed A LOT
- Old configuration styles did not survive nicely
- Some integrations had to be rewritten, not upgraded
- Tests helped, but also exposed how much old magic we had
- AI / OpenRewrite helped with the boring changes



PAINFUL

Hibernate - Summary

- Hibernate was not that bad :)
- Hibernate tightened query and mapping validation.
- Many bugs were fixed.
- Removed deprecated APIs.



OK

Hibernate Search - summary

- There were a lot of usages.
- We have done this on the main branch.
- Searches across the whole application had to be tested, and that was a **huge effort**.
- **(Almost) All indexes** had to be recreated, that complicated our deployment.
- APIs of **6.2.4.Final** are mostly the same as in **7.2.x**, we can postpone upgrade to **7.2.4.Final** to after the migration.

Targets
 Q Occurrences of 'org.hibernate.search.' in Project
 Found occurrences in Project 1,133 results

Upgrade to Hibernate Search 6

Merged requested to merge into master 9 months ago

All threads resolved! Add a to-do item

Overview 3 Commits 1 Pipelines 0 Changes 903

Compare master and latest version 903 files +16108 -16558

Add elasticsearch migration for new indexes

Merged requested to merge into master 3 months ago

Add a to-do item

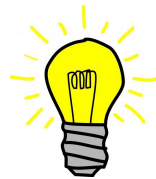
Overview 0 Commits 2 Pipelines 0 Changes 2

Compare master and latest version 2 files +3470 -1

PAINFUL

Important lessons from the skipped pain

- Some migrations are not isolated
- Frameworks depend on each other
- Libraries disappear or change APIs completely
- Old code contains old business assumptions
- Tools help, but they don't understand your production reality
- At some point the question becomes:
“Should we upgrade this, or finally replace it?”





Benefits

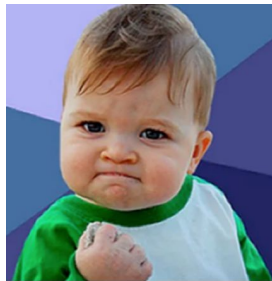
Benefits

Better
resource
utilization

Server startup
cut to 50%

Improved
processes in
company

New features!
Java, Spring,
Hibernate



Better
vulnerability
management

Developer
Experience

Agentic AI vs modernization

On our main branch

- As the main migration started moving, more things were done on main:
- Replace velocity templates with Freemarker templates for emails
- Upgrade to Elasticsearch 7.10 and MySQL 8.4
- Decision made: move authentication to separate service
 - OAuth2
 - Sessions
 - SAML
- Rewrite of some older components
- One of them: **indexing queues rewrite**

The indexer we wrote 14 years ago

- From time to time, our indexing was not really good enough anymore
- We kept it alive for years
- From time to time we needed to patch it
- But at our scale, it started to bother everyone:
 - Some customers were unhappy about search
 - Support was on the edge
 - Developers were not happy



Architecture Decision Record (ADR)

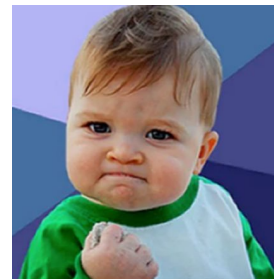
- We wrote an ADR for rewrite of the Indexing Queues
- Architecture Board accepted it
- The plan was to fix it properly, avoiding issues that we currently have

TP-013 - Improve control of priorities, and performance of Indexing Queues

Status	APPROVED
Impact	MEDIUM
Drivers	Maintainability, Flexibility, Fault Tolerance
Approver	Tech Leads

V2 implementation

- I had just learned about Ralph loops
- After trying different AI tools, and good initial results with Opus 4.5, we finally got Claude Code
- I started the reimplementation of the indexer
- Initial results were really good
- I was honestly quite happy



Ralph loops

Command

```
export CLAUDE_CODE_EFFORT_LEVEL=max;  
for i in {1..10}; do cat architecture/v2/prompt.md  
| wclaude --model claude-opus-4-6 -p; done
```

Study `trainingportal/architecture/v2/plan.md` and `trainingportal/architecture/v2/progress.md`, then perform only the next task to be done. The task can be completed only if the relevant tests are green.

prompt.md

After the task:

- update `progress.md`
- commit only the relevant changes when explicitly asked to commit
- report the step number, current time, model name, test runtime, and context usage

My Ralph loops

- **Three files:**
- [prompt.md](#) - the prompt shown on the previous page
- [plan.md](#) - plan made with */plan* and with a human review based on ADR
- [progress.md](#) - the state tracker of the implementation

Keep the session use small

Step 50 — Preserve error state for permanently failed items on re-enqueue. 184 tests pass.	Marek Dominiak	Yesterday 12:59 PM
Step 49 — Error Rate Alarm. 182 tests pass.	Marek Dominiak	Yesterday 12:45 PM
Step 48 — Populate/Re-index All Items. 174 tests pass.	Marek Dominiak	Yesterday 12:21 PM
Step 47 — Replace wildcard imports in test files. 174 tests pass.	Marek Dominiak	Yesterday 7:07 AM
Step 46 — Replace wildcard imports with explicit imports. 174 tests pass.	Marek Dominiak	Yesterday 6:53 AM
Step 45 — Update docs and Postman for Steps 41-44. 174 tests pass.	Marek Dominiak	Yesterday 6:32 AM
Step 44 — Production-ready auto-scale defaults. 174 tests pass.	Marek Dominiak	Yesterday 6:25 AM
Step 43 — Remove expensive state item COUNT queries. 174 tests pass.	Marek Dominiak	Yesterday 5:54 AM
Step 42 — Wall-clock batch time stats. 182 tests pass.	Marek Dominiak	Yesterday 5:40 AM
Step 41 — In-memory staleness check (zero DB queries). 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 5:39 PM
Step 40 — Update docs and Postman collection. 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 1:48 PM
Step 39 DRY review. 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 1:33 PM
Step 37 — Consolidate contract-expand scripts. 211 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 12:39 PM
Step 36 — Improve observability: commit/flush timing + staleness check throttling. (1) Commit/flush timing: Stopwatch wrapping transactionTemplate.execute() captures Hiber	Marek Dominiak	3/10/26, 12:15 PM
Step 35 — Remove autoTuneBatchSize feature (keep autoScaling of consumers). Removed batch size auto-tuning: fields/methods from IndexerProperties, IndexConsumer, Cor	Marek Dominiak	3/10/26, 11:44 AM
Step 34 — Include IndexConfiguration (batchSize) in properties REST endpoints. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 11:36 AM
Step 33 — Add comprehensive Javadoc to IndexerProperties and configuration classes. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 11:28 AM
Step 32 — 'EntityIndexQueueRepositoryImpl.java' — Replaced 'fetchRemainingExcludingOverrides' (NOT IN query) with 'fetchByEntityClassCount' (per-class queries mirroring	Marek Dominiak	3/9/26, 3:13 PM
Step 31 — Soft-delete-aware fetch performance indexes. Add processedAt as leading column to all composite fetch indexes so MySQL can skip soft-deleted rows via index se	Marek Dominiak	3/9/26, 2:05 PM
Step 30 — Eliminate per-entity-class fetch loop. Replace 25+ SELECT FOR UPDATE SKIP LOCKED queries per batch with single NOT IN query. Co-Authored-By: Claude Opus 4.6	Marek Dominiak	3/9/26, 1:53 PM
Step 29 — Entity fetch optimization + status config	Marek Dominiak	3/9/26, 12:23 PM
Step 28 — Documentation & Postman audit	Marek Dominiak	3/6/26, 1:41 PM
Step 27 — Developer FAQ: Group stats lifecycle	Marek Dominiak	3/6/26, 1:22 PM
Step 26 — REST stop-all / start-all endpoints	Marek Dominiak	3/6/26, 1:09 PM
Step 25 — Periodic stats reconciliation	Marek Dominiak	3/6/26, 12:49 PM
Step 24 — Default max consumers 10 → 3	Marek Dominiak	3/6/26, 12:20 PM
Step 23 — Plan for bullet-proofing indexing infrastructure	Marek Dominiak	3/6/26, 12:11 PM
Step 22 — Troubleshooting guide + solution comparison docs sync	Marek Dominiak	3/6/26, 6:51 AM
Step 21 — Reconcile group stats on coordinator startup	Marek Dominiak	3/6/26, 6:42 AM
Step 20 — ADR + Developer FAQ update after in-memory cache	Marek Dominiak	3/6/26, 6:24 AM
Step 19 — Tests for in-memory pendingDecrements cache	Marek Dominiak	3/6/26, 6:12 AM
Step 18 — Wire flushPendingDecrements into IndexCoordinator lifecycle	Marek Dominiak	3/6/26, 6:05 AM
Step 17 — In-memory pendingDecrements cache for decrementGroupStats	Marek Dominiak	3/6/26, 5:53 AM
Step 16 — Plan: in-memory group stats cache to eliminate decrementGroupStats bottleneck	Marek Dominiak	3/6/26, 5:25 AM

What V2 addressed

- Simplifying reliance on unnecessary components
- Prioritization and fairness that we lacked
- Dealing with many duplicates
- Much better monitoring
- REST controllers to manipulate indexers
- Postman collections

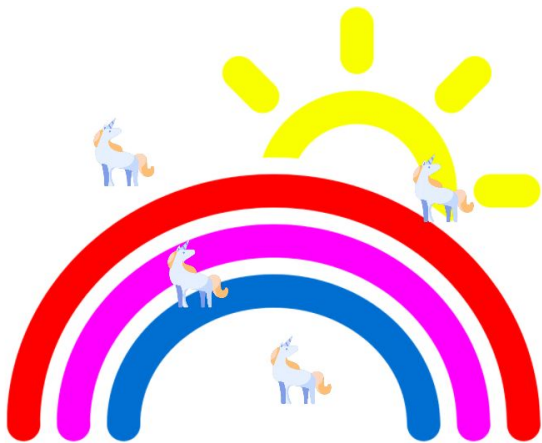


Around 250 Ralph loops later...

- Test coverage close to 100% + performance tests
- I tested heavily
- I had VisualVM profiling sessions
- I was happy to show it to the team
- We put it on beta
- I got feedback and fixed few things
- We went to **staging**
- And shortly after **production**

Merged Marek Dominiak

248 files +23616 -5697



Production challenged my confidence

- The release didn't go as planned
- The production traffic was much higher than anything we tested
- We rolled forward with fixes
- I didn't keep old indexer behind feature flag
- Eventually we needed to rollback
- Lovely 10 hours after work preparing revert to V1
- We stabilized the system
- And finally, we went to sleep



Critical issue

- V2 was not completely wrong
- But production showed us what our tests didn't
- I tried to move V2 forward
- Generated TUI app to see:
 - Elasticsearch
 - Locking
 - Performance of indexing
- But **I still couldn't reproduce the production issue properly**

V3 Indexing TUI refresh 5s host 127.0.0.1 up 8430s 14:40:34

QUEUE					CONSUMERS (REST)			
type	pending	claimed	stale	oldest	type	cons	run	depth
USER	739 (+124)	17	0	29s	USER	1	yes	770
CUR	252 (-12)	300 (+47)	0	7s	CUR	3	yes	233
COURSE	0	0	0	-	COURSE	1	yes	0
RESUME	0	0	0	-	RESUME	1	yes	0
ENTITY	0	0	0	-	ENTITY	1	yes	0
total: 991 pending 317 claimed					started: yes ver: V3 hdx: yes total cons: 7			

DETAIL

ops: UPDATE 991
prio: HIGH 995
top: USER/3614=528 CUR/3614=228 USER/498=201 CUR/498=24 USER/3331=10

ES INDEX			
index	count	d	rate/s
user	341.5K	0	0.0
cur	45.5K	+47	+9.4
course	45.1K	0	0.0
resume	0	0	0.0

LOCKS

row_lock_wait: 16437 (+0) current: 0 avg 234ms max 58397ms deadlocks: 0
long-tr: 0 indexing_queue: 0 dirty_lock_wait: 0 [priv: unavailable]
since start long-tr: 0 lock_wait: 0 deadlocks: 0

q quit r refresh +/- interval p pause

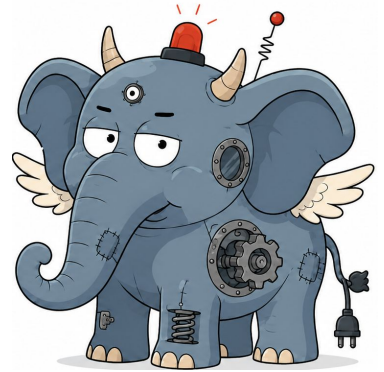
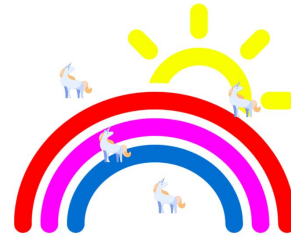
The magical “break system” function

- Someone showed me a magical “break system” functionality
- Finally, I could reproduce the issue locally, at least partially
- But every attempt to avoid locks/slowness in V2 failed
- At some point I was done with V2 ...



What was wrong with V2?

- I generated way too much code
- It became too complicated to deal with
- There were too many heavy tests
- Feedback loops became slower
- I got too many dopamine shots from generating stuff
- Experience told me that the solution should have been MUCH simpler
- I didn't leave any quick rollback option



AI SLOP



Three alternatives

“Generate at least three alternatives.” - Eric Evans

V3 implementation

- I updated the ADR
- I created several plans with all models I had access to
- Cross-checked them with myself and against each other
- One plan caught my eye
- V3 MUST BE:
 - Quite **SIMPLE**
 - As close to no-lock implementation as possible
 - Behind a feature flag
 - With old V1 indexer still available for quick rollback

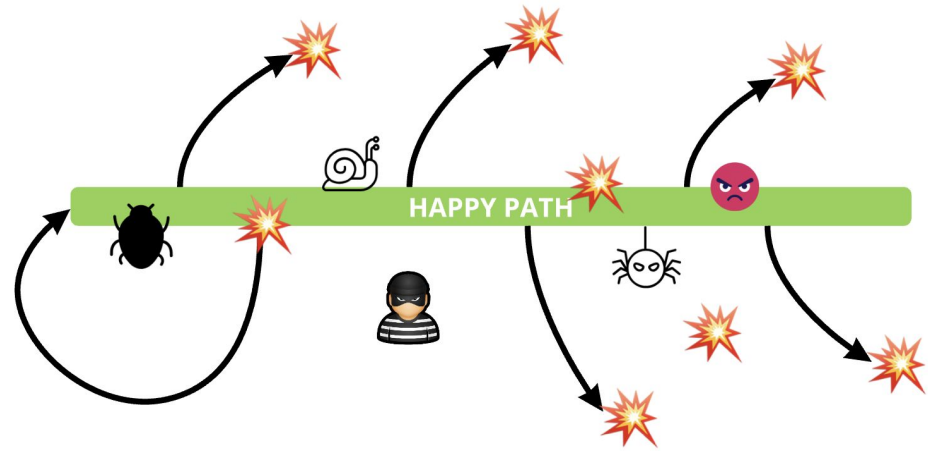


VS



This time I restrained myself

- I didn't let the implementation grow
- I didn't let the tests become slow
- I accepted more trade-offs
- I documented those trade-offs
- I documented potential optimizations for the future
- My mindset was vary of potential issues



Occam's
Razor

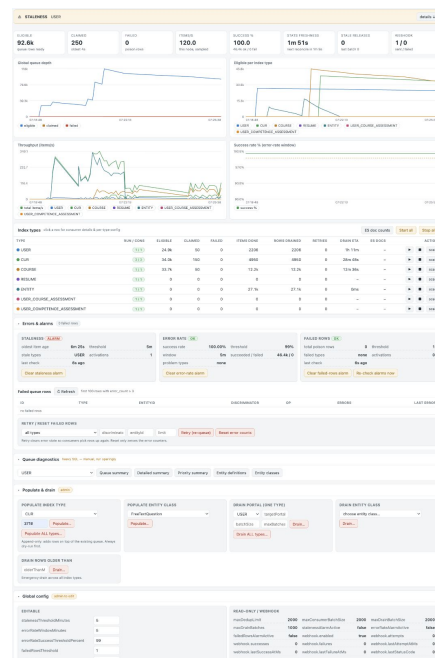
Bulletproofing V3

- Each risky part of indexing got extra attention
- “Break system” must not reproduce V2 issues
- Tested heavily again
- Showed it to the team
- We put it on beta
- Generated one HTML + JS dashboard for manipulation/monitoring
- For those interested: V3 uses the Claim-Check Pattern

[Merged](#) Marek Dominiak requested to merge [indexing-v3](#) into [master](#) 3 weeks ago

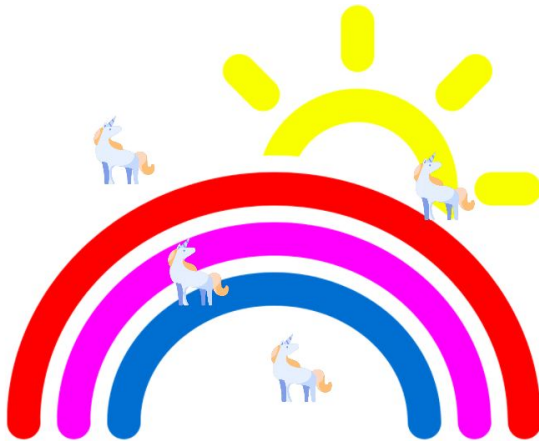
133 files +15996 -23241

44 files +7546 -428



Release expectation

On 16th we turn on the flag and we expect



In the worst case, we have a very quick rollback option

Lessons learned

AI needs a harness

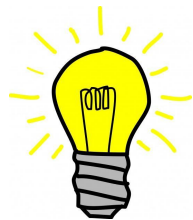
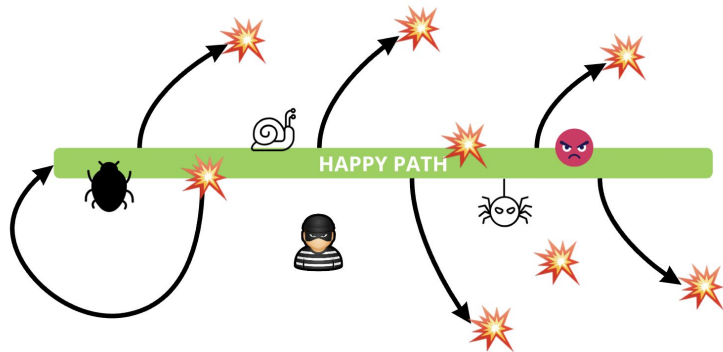
- Monitoring
- Traceability
- Verification
- Sensible test coverage
- Profiling sessions
 - Async profiler
 - VisualVM
- Human verification at some point
- Feature flag / rollback path

✓ Tests passed: 1,075,



Enterprise AI models are not as stable as I would like

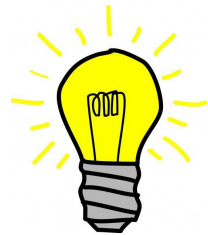
- Models change their performance
- Models are deprecated
- Thinking time can change
- Access can change (e.g. Fable 5 example)
- Tool wrappers can make models behave differently (Claude code vs API)
- Vendors, platforms, or regulation can change access anytime
- So don't build your whole process around one magical model



- Resourcing models are indicated by a lightbulb icon

Agentic AI is a double-edged sword

- It can help you remove tech debt and make progress
- Or you can generate AI slop and create even more tech debt
- AI is a multiplier of your capabilities
 - Wrong direction → you move away from the target 100x faster
 - Right direction → you move closer 100x faster
- Keep human in the loop every few iterations



Don't allow to accumulate Tech Debt

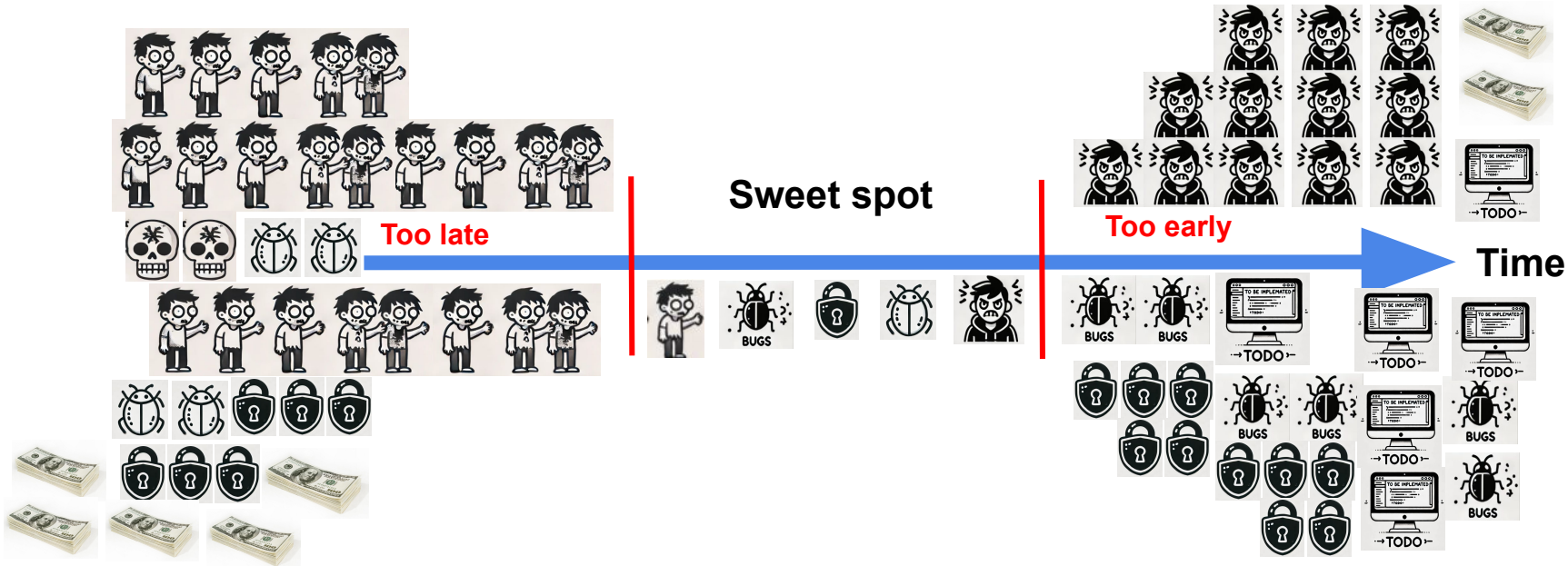
Avoid those “strategies” to deal with Tech Debt:



Find your sweet spot for the Upgrade window

The Sweet spot for the Upgrade window

- Find your sweet spot between lagging behind and upgrading too early.
- Martin Fowler's advice: *"If it's painful, do it more often"*.



Q/A

Thank You!

Presentation Slides

 **marekdominiak**

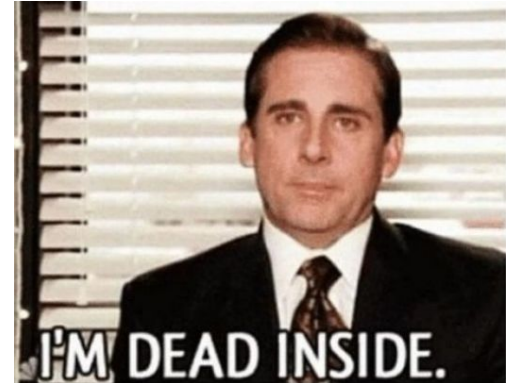


Appendix:

Challenge 3 - Changed APIs

Changed APIs

- javax -> jakarta
- Hibernate, Hibernate Search
- Spring Security Core
- Infinispan
- Mockito (we removed PowerMock and replaced it using Mockito features)
- Apache Tiles
- Spring-Webflow
- Apache HttpClient, Apache HttpCore



javax.* -> jakarta.*

javax.* -> jakarta.*

- It is tougher than it seems
- APIs that we had to replace
 - Jakarta Persistence API
 - Jakarta Servlet API
 - Jakarta Mail API
 - Jakarta Validation API
 - Jakarta Annotation API
 - Jakarta Transaction API
 - Jakarta EL API
 - Jakarta JMS API
- The hardest were:
 - Jakarta Servlet API
 - Jakarta Persistence API

Jakarta Servlet API

- A lot of usages

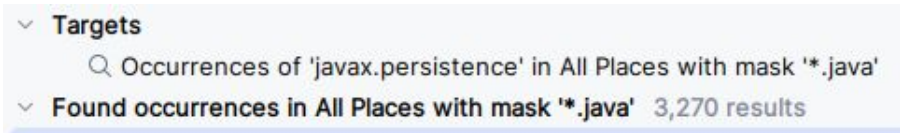


▼ Targets
🔍 Occurrences of 'javax.servlet' in Project with mask '*.java'
▼ Found occurrences in Project with mask '*.java' 858 results

- Servlet API being a transitive dependency in a lot of our dependencies.
 - This meant we needed to upgrade them ...
 - To name some:
 - Hystrix
 - CXF
 - HttpClient, HttpCore
 - and so on ...

Jakarta Persistence API

- A lot of usages



- Some of the libraries have duplicate versions for jakarta and javax!
- `infinispan-core` for javax
- `infinispan-core-jakarta` for jakarta
- which caught us by surprise.

[Home](#) » [org.infinispan](#) » infinispan-core



Infinispan Core

Infinispan core module

[Home](#) » [org.infinispan](#) » infinispan-core-jakarta



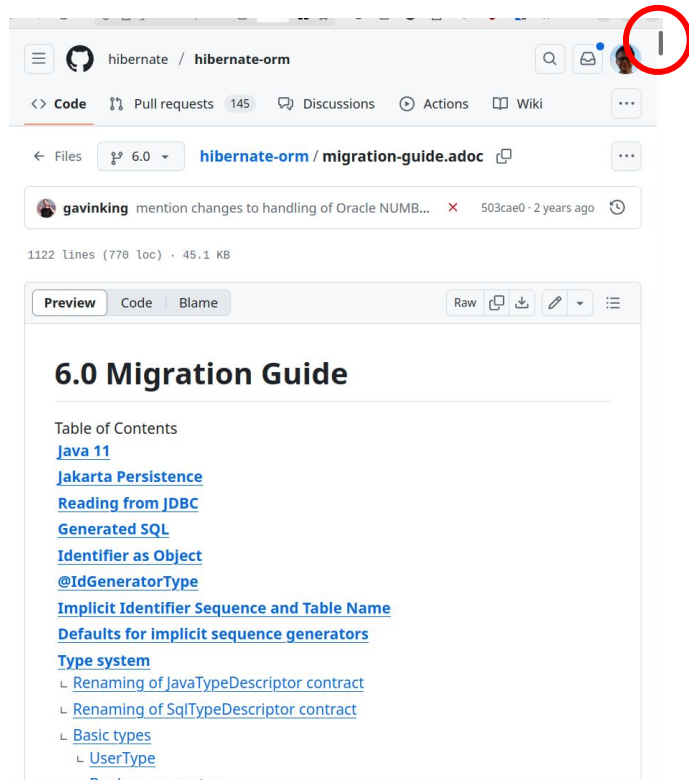
Infinispan Core Jakarta EE

Infinispan core module for Jakarta EE

Hibernate 1/6



- Where do I even start?
- There is this mythical thing called documentation.
- Migration guide is your friend
<https://github.com/hibernate/hibernate-orm/blob/6.0/migration-guide.adoc#instant-mapping-changes>



Hibernate 2/6

- Deprecated Criteria API -> JPA Criteria API

Old:

```
private Long getCoursePaymentsCount(Course course) {
    Criteria criteria = getCriteria(CoursePayment.class);
    criteria.add(eq(FieldName.course.toString(), course));
    criteria.add(isNotNull(FieldName.dateOfPayment.toString()));

    return (Long) criteria.setProjection(Projections.rowCount())
        .uniqueResult();
}
```

```
private Criteria getCriteria(Class clazz) {
    return getHibernateSession().createCriteria(clazz);
}

private Session getHibernateSession() {
    final Session session =
        (org.hibernate.Session) entityManager.getDelegate();
    return session;
}
```

New:

```
private Long getCoursePaymentsCount(Course course) {
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Long> query = cb.createQuery(Long.class);
    Root<CoursePayment> root = query.from(CoursePayment.class);
    query.select(cb.count(root));
    query.where(cb.equal(root.get(FieldName.course.toString()), course),
        cb.isNotNull(root.get(FieldName.dateOfPayment.toString())));
    return entityManager.createQuery(query).getSingleResult();
}
```

Hibernate 3/6

- Changes in Mappings:
 - Ref: <https://github.com/hibernate/hibernate-orm/blob/6.0/migration-guide.adoc#instant-mapping-changes>
 - In Hibernate 5.7 this mapping:
Entity:

```
@Temporal(TemporalType.TIMESTAMP)  
private Date appliedOn;
```


Table:

```
`appliedOn` DATE
```


worked without any issues.
 - It didn't work in Hibernate 6.x, so we decided to change the type on the DB level
Table:

```
`appliedOn` DATETIME(6)
```
 - We decided to set:

```
<prop key="hibernate.type.preferred_instant_jdbc_type">DATE</prop>
```

Hibernate 4/6

- In Hibernate 5.7 mapping like:

`@Enumerated`

```
private QuestionAttachmentStatus questionAttachmentStatus;
```

could be mapped to an INT column.

- Now it needed to be mapped to TINYINT.
- What is good as it makes our system a tiny bit more optimal.

Hibernate 5/6

- Stronger validation of native queries.
- Native queries return Long instead of BigInteger

Old:

```
return ((BigInteger) query.getSingleResult());
```

New:

```
return ((Long) query.getSingleResult());
```

- Better validation of native queries for entities with secondary tables

Old:

```
Query query = entityManager.createNativeQuery(  
    "SELECT c.*, crp.* FROM CourseRegistration c " +  
    " LEFT JOIN courseregistration_price crp ON crp.id = c.id", CourseRegistration.class);
```

New:

```
Query query = entityManager.createNativeQuery(  
    "SELECT c.*, crp.currency_id, crp.amount FROM CourseRegistration c " +  
    " LEFT JOIN courseregistration_price crp ON crp.id = c.id)", CourseRegistration.class);
```


Hibernate 6/6

- Identifier generation default has been changed

As of 6.0, Hibernate by default creates a sequence per entity hierarchy instead of a single sequence

`hibernate_sequence` .

Due to this change, users that previously used `@GeneratedValue(strategy = GenerationType.AUTO)` or simply `@GeneratedValue` (since `AUTO` is the default), need to ensure that the database now contains sequences for every entity, named `<entity name>_seq` . For an entity `Person` , a sequence `person_seq` is expected to exist. It's best to run `hbm2ddl` (e.g. by temporarily setting `hbm2ddl.auto=create`) to obtain a list of DDL statements for the sequences.

- We decided to depend on our Mysql to generate Identifiers.

From:

```
@Id
@GeneratedValue
private Integer id;
```

To:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer id;
```

Hibernate - Summary

- Hibernate solidified their query and mapping validation.
- Many bugs were solved.
- Removed deprecated APIs.



Hibernate Search - plan

- We wanted to be able to use newer versions of Elasticsearch.

Hibernate Search Version	Compatible Elasticsearch Version
5.9.3.Final	2.0 – 5.6
6.2.4.Final	7.10 – 8.12
7.2.2.Final	8.14 – 8.15

- **Plan:**

- 5.9.3.Final (Lucene API supported, default backend)

- -> **6.2.4.Final - main branch**
(Lucene types removed from public API,
Hibernate 6.2.+, Elasticsearch: 7.x.+)

- -> **7.2.2.Final - upgrade branch**
(API is fully backend-agnostic,
Hibernate 6.6.+, Elasticsearch: 8.x.+)

Hibernate Search Version	Compatible Hibernate ORM Versions
5.9.3.Final	5.2.3.Final – 5.2.x
6.2.4.Final	6.2.24.Final
7.2.2.Final	6.6.0.Final

Hibernate Search - 1/6

- Start with: <https://hibernate.org/search/documentation/migrate/>
- 5.x -> 6.x: https://docs.jboss.org/hibernate/search/6.0/migration/html_single/

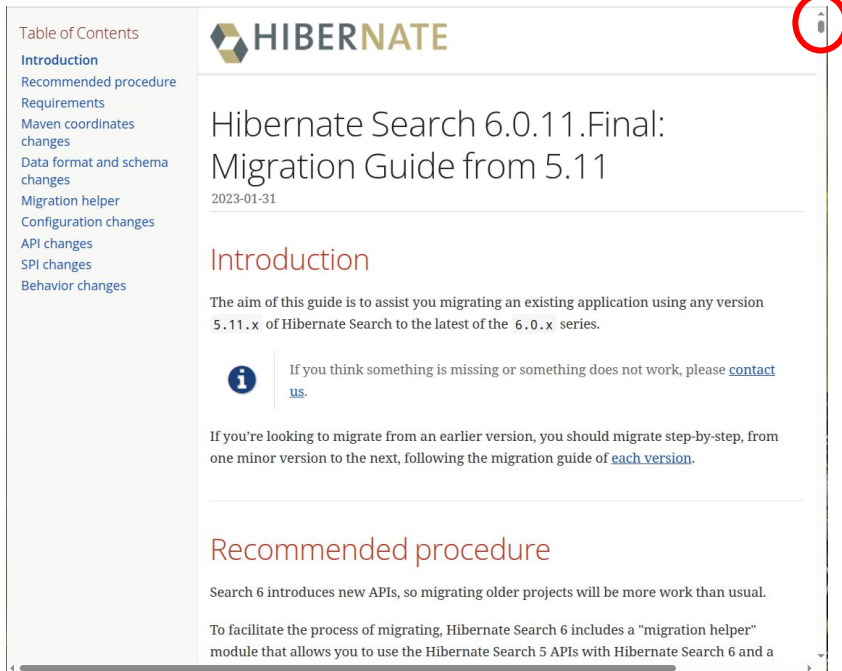


Table of Contents

- Introduction
- Recommended procedure
- Requirements
- Maven coordinates changes
- Data format and schema changes
- Migration helper
- Configuration changes
- API changes
- SPI changes
- Behavior changes

HIBERNATE

Hibernate Search 6.0.11.Final: Migration Guide from 5.11

2023-01-31

Introduction

The aim of this guide is to assist you migrating an existing application using any version 5.11.x of Hibernate Search to the latest of the 6.0.x series.

 If you think something is missing or something does not work, please [contact us](#).

If you're looking to migrate from an earlier version, you should migrate step-by-step, from one minor version to the next, following the migration guide of [each version](#).

Recommended procedure

Search 6 introduces new APIs, so migrating older projects will be more work than usual.

To facilitate the process of migrating, Hibernate Search 6 includes a "migration helper" module that allows you to use the Hibernate Search 5 APIs with Hibernate Search 6 and a

Hibernate Search - 2/6

- Mapping Annotation changes

- Old:

```
@org.hibernate.search.annotations.Indexed
public class ContentDocument {
    // ...
    @Field(name = "contentType")
    @SortableField(forField = "contentType")
    @FieldBridge(impl = ContentTypeSortFieldBridge.class)
    public ContentType getType() {
        return ContentType.ContentDocument;
    }
}
```

- New:

```
@org.hibernate.search.mapper.pojo.mapping.definition.annotation.Indexed
public class ContentDocument {
    // ...
    @Field(name = "contentType")
    @IndexingDependency(reindexOnUpdate = ReindexOnUpdate.NO)
    public ContentType getType() {
        return ContentType.ContentDocument;
    }
}
```

Hibernate Search - 3/6

- Annotation changes (Ref: https://docs.jboss.org/hibernate/search/6.0/migration/html_single/)

Summary: Don't read it :) Use when needed

1. **Complete rewrite of Bridge API.**
2. **Basic Annotations:**
 - `@org.hibernate.search.annotations.Indexed` → `@org.hibernate.search.mapper.pojo.mapping.definition.annotation.Indexed`
 - `@Field` → Split into multiple specialized annotations:
 - `@FullTextField`: For analyzed text fields (full-text search)
 - `@KeywordField`: For non-analyzed fields (exact matching, sorting, aggregations)
 - `@GenericField`: For non-text fields (numbers, dates, etc.)
3. **Field Customization:**
 - `@SortableField` → `sortable = Sortable.YES` parameter in field annotations
 - `@FieldBridge` → `@ValueBridgeRef` OR `@TypeBinderRef`
 - `@Analyzer` → `analyzer` parameter in `@FullTextField`
4. **New Annotations:**
 - `@IndexingDependency`: Controls when fields are reindexed
 - `@ObjectPath` and `@PropertyValue`: Define paths to dependent properties
 - `@TypeBinding`: Binds a custom type to an entity

Hibernate Search - 4/6

- Indexing API changes:

```
FullTextSession fullTextSession = prepareFullTextSession();  
fullTextSession.index(entityToCreateIndex);
```



```
SearchSession searchSession = getSearchSession();  
searchSession.indexingPlan().addOrUpdate(entityToCreateIndex);
```

Hibernate Search - 5/6

- Search API changes:

```
FullTextEntityManager fullTextEntityManager = Search.getFullTextEntityManager(entityManager);
QueryBuilder queryBuilder = fullTextEntityManager.getSearchFactory()
    .buildQueryBuilder().forEntity(Course.class).get();
Query luceneQuery = queryBuilder.keyword().onField("title")
    .matching(searchTerm).createQuery();
FullTextQuery fullTextQuery = fullTextEntityManager.createFullTextQuery(luceneQuery, Course.class);
```



```
SearchSession searchSession = org.hibernate.search.mapper.orm.Search.session(entityManager);
SearchResult<Course> result = searchSession.search(Course.class)
    .where(f -> f.match().field("title").matching(searchTerm)).fetch(20);
```


Hibernate Search - 6/6

- Key Search API changes:
- `FullTextEntityManager` → `SearchSession`
- `QueryBuilder` → Lambda-based search DSL
- `FullTextQuery` → `SearchResult`
- New fluent API for defining queries



Hibernate Search - summary

- There were a lot of usages.
- We have done this on the main branch.
- Searches across the whole application had to be tested, and that was a **huge effort**.
- **(Almost) All indexes** had to be recreated, that complicated our deployment.
- APIs of **6.2.4.Final** are mostly the same as in **7.2.x**, we can postpone upgrade to **7.2.4.Final** to after the migration.

▼ Targets
 🔍 Occurrences of 'org.hibernate.search.' in Project
 ▼ Found occurrences in Project 1,133 results

Upgrade to Hibernate Search 6 Edit Code ▼ ⋮
 Merged requested to merge into master 9 months ago
 All threads resolved! Add a to-do item
 Overview 3 Commits 1 Pipelines 0 **Changes 903**
 Compare master ▼ and latest version ▼ **903 files +16108 -16558**

Add elasticsearch migration for new indexes Edit Code ▼ ⋮
 Merged requested to merge into master 3 months ago
 Add a to-do item
 Overview 0 Commits 2 Pipelines 0 **Changes 2**
 Compare master ▼ and latest version ▼ **2 files +3470 -1**

Spring security 1/4

- Spring Security Core (3.x -> 6.x) - changes in:

- Authentication providers
- Password encoders
- XML / Java based configuration
- CSRF filters
- Authorization

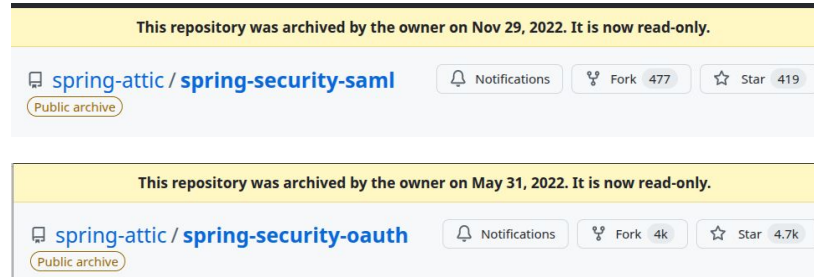
```
<sec:authorize ifAllGranted="ROLE_ADMIN,ROLE_USER">  
    <p>Must have ROLE_ADMIN and ROLE_USER</p>  
</sec:authorize>
```



```
<sec:authorize access="hasRole('ROLE_ADMIN') and hasRole('ROLE_USER')">  
    <p>Must have ROLE_ADMIN and ROLE_USER</p>  
</sec:authorize>
```

- And all would be fine if those were the only changes ...

Spring security 2/4



... needed **complete** rewrite, around 7000 LOC to rewrite



Spring security 3/4

How to assess effort needed to upgrade of Spring Security and related libraries?

Simple heuristic:

The higher the number of classes with name starting with “Custom” related to Spring Security you have, the harder it will be.

All Classes Fi

Q- security.Custom

- CustomAuthenticati
- CustomScopeVoter
- CustomTokenEnhar
- CustomTokenServic
- CustomJdbcTokenS
- CustomScopeVoter
- CustomJdbcTokenS
- CustomSAMLBootst
- CustomSAMLEntryF
- CustomAccessDeci
- CustomRoleVoter
- CustomSessionMan
- CustomSAMLMetad
- CustomSAMLProce
- CustomSAMLConte
- CustomSAMLUserD
- CustomSAMLRelayS
- CustomSAMLUserD
- CustomSimpleUrlAu

- CustomHttpSessionReques
- CustomSamlSimpleUrlAuthe
- CustomSavedRequestAware
- CustomConcurrentSessionF
- CustomInvalidSessionStrate
- CustomConcurrentSessionC
- CustomAccessDeniedHandl
- CustomHttp401Unauthentic
- CustomPreAuthenticatedGr
- CustomSimpleUrlAuthentic
- CustomHttpSessionReques
- CustomTokenServicesInteg
- OAuth2CustomRequestPara
- CustomUnanimousBasedAc
- CustomUserDetailsService
- CustomUserDetailsService
- CustomTwoFactorAuthentic
- CustomUsernamePassword
- CustomUserDetailsService

Spring security 4/4

This is still a **Work In Progress** ...

Springfox -> Springdoc 1/2

- Springfox has never been recognized by the Spring team, but there was no alternative.
- Finally, we have got springdoc-openapi and spring-hateoas 2.

[Stack upgrade] Migrate to springdoc-openapi, spring-hateoas 2 and spring-data 4

Merged  requested to merge  into `feature/tp-stack-upgrade` 2 weeks ago

Overview 8 Commits 2 Pipelines 0 Changes 1225

All threads resolved! Add a to-do item

Compare `feature/tp-stack-u...` and latest version

1225 files +6620 -6802

Springfox -> Springdoc 2/2

- Mainly, simple annotation changes.
- The whole configuration had to be changed.
- **Expected** changes in **generated OpenAPI documentation**, that can lead to **different generated TypeScript types** used on Frontend applications.

This is still a **Work In Progress** ...

Infinispan

- Migration from 8.x to 14.x
- The whole xml configuration had to be changed.

```
<invalidation-cache name="enrollments">
```

```
<eviction size="5000000" strategy="LIRS" type="COUNT"/>
```

```
<expiration max-idle="-1" lifespan="1800000" interval="5000"/>
```

```
</invalidation-cache>
```

```
44
```

```
42
```

```
45
```

```
43
```

```
46
```

```
44
```

```
47
```

```
45
```

```
48
```

```
46
```

```
<invalidation-cache name="enrollments">
```

```
<memory max-count="5000000" when-full="REMOVE"/>
```

```
<expiration max-idle="-1" lifespan="1800000" interval="5000"/>
```

```
</invalidation-cache>
```

- Finally better control over **the memory use**.
- We expect some issues once deploying to AWS.
- All in all, not that hard but remember, that there are two artifacts in Mvn repo.

[Home](#) » [org.infinispan](#) » [infinispan-core](#)



Infinispan Core

Infinispan core module

[Home](#) » [org.infinispan](#) » [infinispan-core-jakarta](#)



Infinispan Core Jakarta EE

Infinispan core module for Jakarta EE

Some smaller API changes

- MockServer → WireMock
- PowerMock → Mockito
- Apache Tiles → removed
- Apache HttpClient, Apache HttpCore
- many others ...

Last notes

- Remember, upgrading a Monolith is hard work.
- Neither Spring nor Hibernate are bad, accumulating Technical Debt is!
- You don't need to do anything, Upgrades and Tech Debt will find you at some point :)
- Having a good tests coverage helps a lot.



Suggestions

- Include as many people from the team to help with migration.
- Spend effort on planning migration.
- Require your team to keep versions of your frameworks/libs up to date.
- Add recurring tasks for keeping upgrades in check.
Automate security upgrades if possible.
- Improve processes in your workplace.

